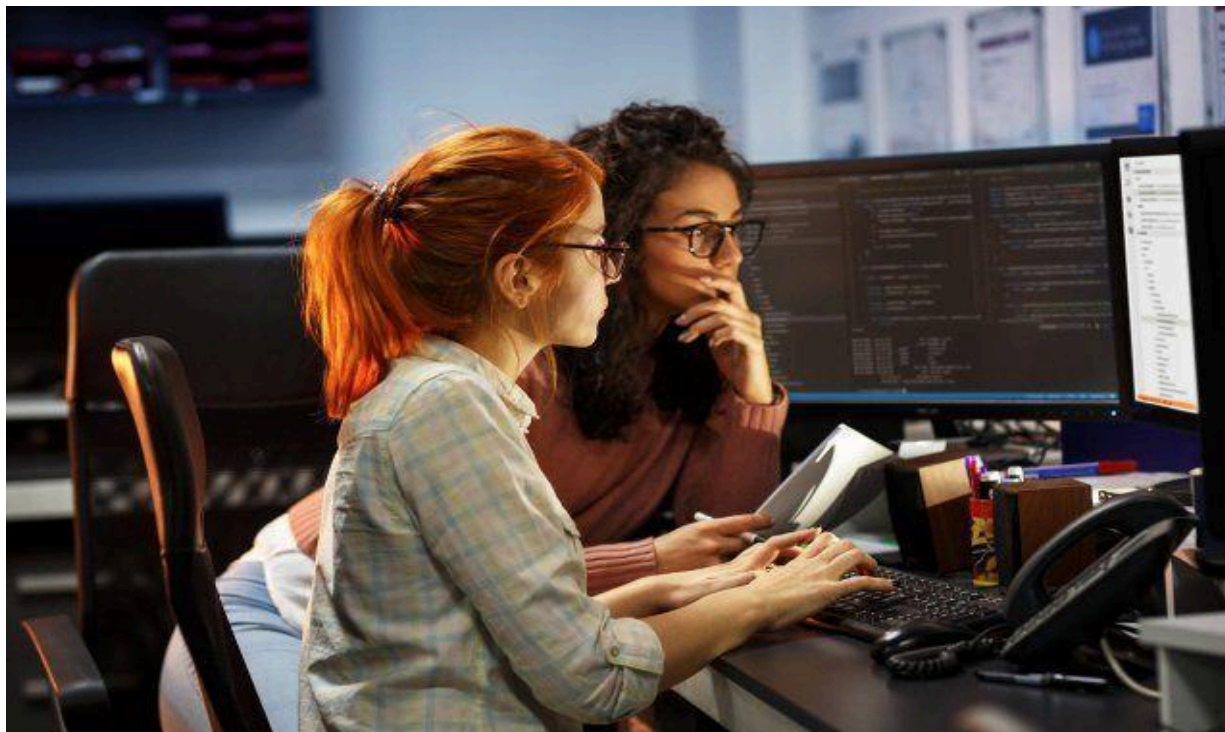




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Programación Orientada a Objetos	Unidad #: 1	Tema: Programación y Principios de Diseño Orientado a Objetos.
	SEMANA 2	
		Autor:

Introducción:

Bienvenidos al material de lectura #02 de la Unidad #1 de la materia Programación Orientada a Objetos. En este documento exploramos la introducción y los conceptos generales de Programación Orientada a Objetos y nos enfocaremos en la estructura general de un programa y sentencias de control.

Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en las actividades propuestas, ya que contribuirán significativamente a tu proceso de aprendizaje.

[illegible]

```
public class Coche {  
    String marca; // Atributo  
  
    public Coche(String marca) { // Constructor  
        this.marca = marca;  
    }  
  
    public void arrancar() { // Método  
        System.out.println("El " + this.marca + " ha  
arrancado.");  
    }  
}  
  
// Archivo: ProgramaPrincipal.java  
// --- CLASE 2: Contiene el punto de entrada del programa ---  
public class ProgramaPrincipal {  
  
    // --- MÉTODO PRINCIPAL (main) ---  
    public static void main(String[] args) {  
        // 1. El programa comienza aquí.  
        System.out.println("Iniciando el programa...");  
  
        // 2. Se crea un objeto (instancia) de la clase  
Coche.  
        Coche miCoche = new Coche("Toyota");  
    }  
}
```

// 3. Se interactúa con el objeto llamando a su método.

```
miCoche.arrancar();
```

// 4. El programa termina cuando finaliza el método main.

```
System.out.println("Programa finalizado.");
```

```
}
```

```
}
```

En esta estructura, `ProgramaPrincipal` actúa como el director de orquesta, creando y utilizando los objetos definidos en otras clases (`Coche`) para que el programa cumpla su función.

Tipos de Operadores en Java

1. Operadores Aritméticos:
 - Se usan para realizar operaciones matemáticas básicas.
 - Ejemplos: `+` (suma), `-` (resta), `*` (multiplicación), `/` (división), `%` (módulo o residuo).
2. Operadores de Asignación:
 - Se usan para asignar un valor a una variable.
 - Ejemplos: `=` (asignación simple), `+=` (suma y asigna), `-=` (resta y asigna), `*=` (multiplica y asigna).
3. Operadores Relacionales (o de Comparación):
 - Se usan para comparar dos valores y el resultado es siempre un valor booleano (`true` o `false`).
 - Ejemplos: `==` (igual a), `!=` (diferente de), `>` (mayor que), `<` (menor que), `>=` (mayor o igual que), `<=` (menor o igual que).
4. Operadores Lógicos:
 - Se usan para combinar dos o más expresiones booleanas.
 - Ejemplos: `&&` (AND lógico, "y"), `||` (OR lógico, "o"), `!` (NOT lógico, "no").
5. Operadores Unarios:
 - Requieren un solo operando para realizar una operación.

- Ejemplos: `++` (incremento), `--` (decremento), `-` (negación aritmética).
6. Operador Ternario:
- Es una forma abreviada de una sentencia `if-else`.
 - Sintaxis: `condición ? valor_si_verdadero : valor_si_falso;`

1.5 Revisión de Sentencias de Control

Las sentencias de control son las herramientas que nos permiten dirigir el flujo de ejecución de nuestro programa. Determinan qué líneas de código se ejecutan, en qué orden y cuántas veces. Sin ellas, un programa solo podría ejecutar una lista de instrucciones de principio a fin, una sola vez.

Existen tres categorías fundamentales de sentencias de control.

1.5.1 Secuenciales

Esta es la forma más simple de control. Las sentencias se ejecutan en orden secuencial, una después de la otra, tal como están escritas en el código. No hay saltos ni repeticiones. La ejecución fluye de arriba hacia abajo.

Ejemplo:

```
// La ejecución es estrictamente secuencial.
```

```
// 1º se declara la variable 'a'.
```

```
int a = 5;
```

```
// 2º se declara la variable 'b'.
```

```
int b = 10;
```

```
// 3º se calcula la suma.
```

```
int suma = a + b;
```

```
// 4º se muestra el resultado en consola.
```

```
System.out.println("El resultado es: " + suma);
```

1.5.2 Selección (o Condicionales)

Las sentencias de selección permiten que el programa tome decisiones.

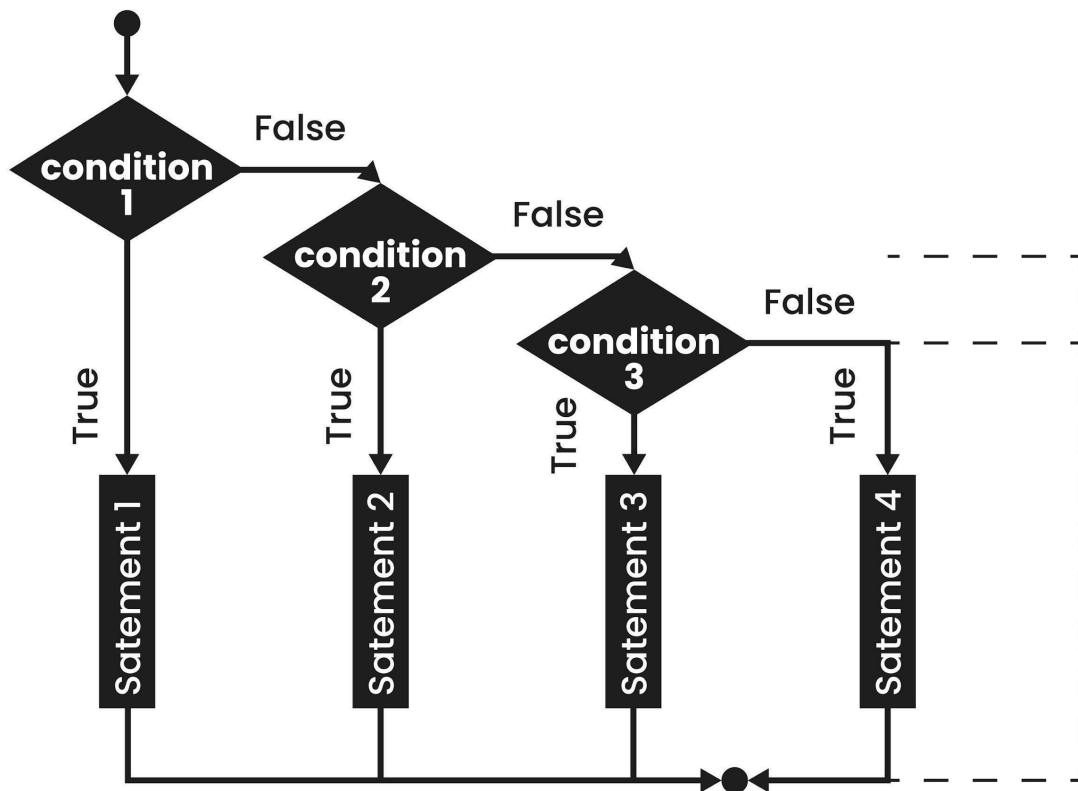
Basándose en una condición que evalúa como verdadera (**true**) o falsa (**false**), el programa elige qué bloque de código ejecutar.

if / else / else if

- **if**: Ejecuta un bloque de código si una condición es verdadera.
- **else**: Ejecuta un bloque de código alternativo si la condición del **if** es falsa.
- **else if**: Permite encadenar múltiples condiciones. Se evalúa solo si las condiciones anteriores fueron falsas.



IF-ELSE-IF STATEMENT



Ejemplo:

```
int edad = 18;
```

```
if (edad >= 18) {
```

```
    // Este bloque se ejecuta porque la condición (18 >= 18)
    es verdadera.
```

```
    System.out.println("Es mayor de edad.");
```

```
} else {
```

```
// Este bloque se ignora.  
System.out.println("Es menor de edad.");  
}  
  
// Ejemplo con else if  
int calificacion = 85;  
  
if (calificacion >= 90) {  
    System.out.println("Excelente (A)");  
} else if (calificacion >= 80) {  
    // Este bloque se ejecuta porque 85 >= 80 es verdadero.  
    System.out.println("Muy Bien (B)");  
} else {  
    System.out.println("Necesita mejorar.");  
}  
  
switch
```

La sentencia `switch` es una alternativa al `if-else` cuando se necesita comparar una variable contra múltiples valores constantes. Es más limpia y legible en esos casos.

Ejemplo:

```
int diaDeLaSemana = 4; // 1=Lunes, 2=Martes, ...  
String nombreDelDia;  
  
switch (diaDeLaSemana) {  
    case 1:  
        nombreDelDia = "Lunes";
```



```
break; // El 'break' es crucial para salir del  
switch.
```

```
case 2:
```

```
    nombreDelDia = "Martes";
```

```
    break;
```

```
case 3:
```

```
    nombreDelDia = "Miércoles";
```

```
    break;
```

```
case 4:
```

```
    nombreDelDia = "Jueves"; // El programa salta  
    directamente aquí.
```

```
    break;
```

```
case 5:
```

```
    nombreDelDia = "Viernes";
```

```
    break;
```

```
default: // Se ejecuta si ningún 'case' coincide.
```

```
    nombreDelDia = "Fin de semana";
```

```
    break;
```

```
}
```

```
System.out.println("Hoy es " + nombreDelDia); // Salida: Hoy  
es Jueves
```

1.5.3 Repetición (o Bucles)

Las sentencias de repetición, también conocidas como bucles o ciclos, permiten ejecutar un bloque de código múltiples veces mientras una condición se mantenga verdadera. Son esenciales para tareas repetitivas.

while

El bucle `while` (mientras) repite un bloque de código mientras su condición de entrada sea verdadera. La condición se comprueba antes de cada iteración. Si la condición es falsa desde el principio, el bucle nunca se ejecuta.

Ejemplo:

```
int contador = 0;

// Mientras (while) el contador sea menor que 5...
while (contador < 5) {
    System.out.println("El contador vale: " + contador);
    contador++; // ¡Importante! Incrementar el contador para
    evitar un bucle infinito.
}

// Salida:
// El contador vale: 0
// El contador vale: 1
// El contador vale: 2
// El contador vale: 3
// El contador vale: 4
```

do-while

El bucle `do-while` es similar al `while`, pero con una diferencia clave: la condición se comprueba después de cada iteración. Esto garantiza que el bloque de código se ejecute al menos una vez, incluso si la condición es falsa desde el principio.

Ejemplo:

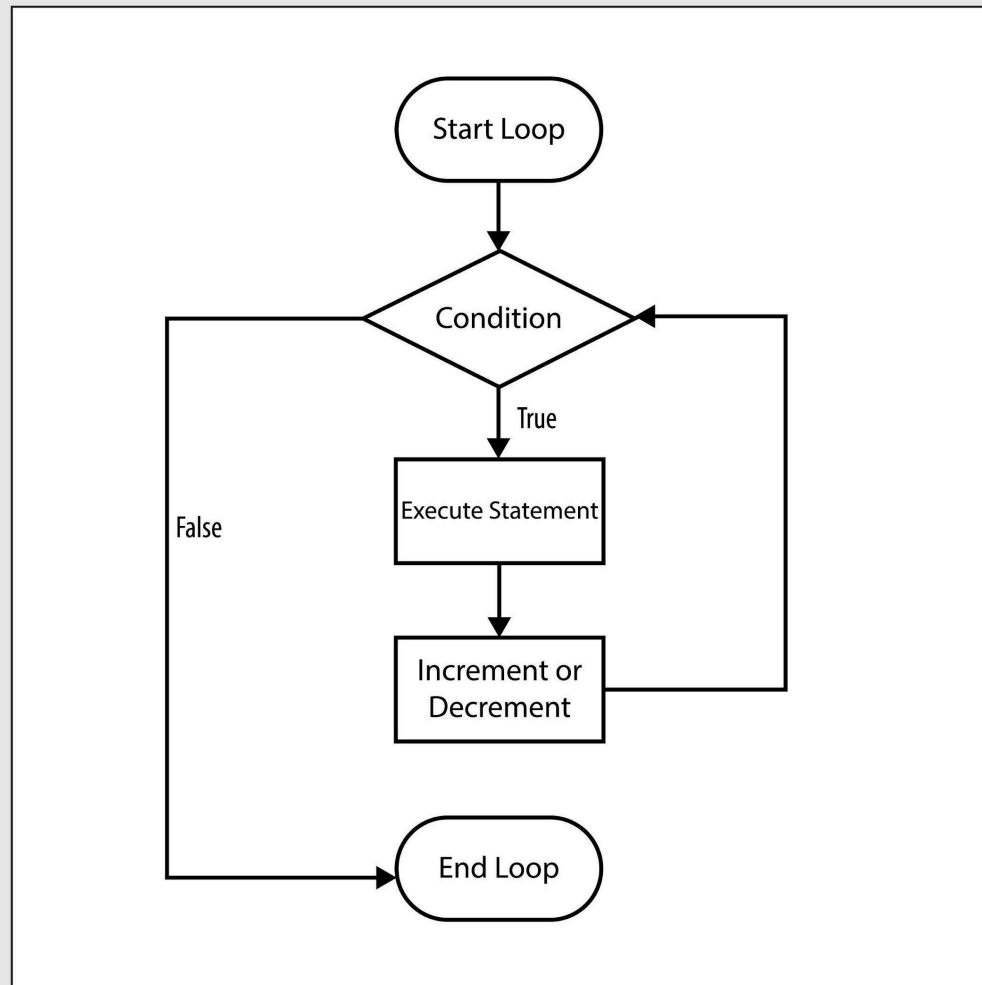
```
int contador = 10;
```

```
do {  
    // Este bloque se ejecuta una vez...  
    System.out.println("Este mensaje se muestra al menos una  
vez.");  
    contador++;  
} while (contador < 5); // ...y luego se comprueba la  
condición (11 < 5 es falso) y el bucle termina.
```

for

El bucle **for** es ideal para cuando se sabe de antemano cuántas veces se quiere repetir una acción. Es más compacto que un **while** porque integra en una sola línea la inicialización de una variable de control, la condición de permanencia y el paso (incremento o decremento).





Ejemplo:

```
// for (inicialización; condición; paso)
for (int i = 0; i < 5; i++) {
    System.out.println("Iteración número: " + i);
}

// Salida:
// Iteración número: 0
```

```
// Iteración número: 1  
// Iteración número: 2  
// Iteración número: 3  
// Iteración número: 4
```

Tipo	Uso principal	Evalúa condición antes	Evalúa condición después
<code>if</code> , <code>switch</code>	Decisiones / selección	✓	✗
<code>while</code>	Repeticiones con condición inicial	✓	✗
<code>do-while</code>	Repeticiones que se ejecutan al menos una vez	✗	✓
<code>for</code>	Repeticiones con contador	✓	✗
<code>for-each</code>	Iterar estructuras de datos	Implícito	✗